

# Assignment 3: Unsupervised Learning — Vancouver Business Licences Explorer

```
import numpy as np
import pandas as pd
import geopandas as gpd
import matplotlib.pyplot as plt
import plotly.express as px
from sklearn.cluster import KMeans
from sklearn.cluster import DBSCAN
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.decomposition import PCA
```

## Part A — Business-Level Clustering

### A1 — Data acquisition and cleaning

```
# load geojson file
data = gpd.read_file("business-licences.geojson")
print(f"Shape: {data.shape}")
data.head()
```

Shape: (204301, 26)

```
{"type": "dataframe", "variable_name": "data"}
```

```
# inspect structure
print("Column dtypes:\n", data.dtypes)
print("\nGeometry types:\n",
data.geometry.type.value_counts(dropna=False))
print("\nCRS:", data.crs)
```

Column dtypes:

|                       |                     |
|-----------------------|---------------------|
| folderyear            | object              |
| licencersn            | object              |
| licencenumber         | object              |
| licencerevisionnumber | object              |
| businessname          | object              |
| businessradename      | object              |
| status                | object              |
| issuedate             | datetime64[ms, UTC] |
| expiredate            | datetime64[ms]      |
| businessstype         | object              |
| businesssubtype       | object              |
| unit                  | object              |
| unittype              | object              |

```

house      object
street     object
city       object
province   object
country    object
postalcode object
localarea  object
numberofemployees float64
feepaid     float64
extractdate datetime64[ms, UTC-07:00]
geom        object
geo_point_2d object
geometry    geometry
dtype: object

```

```

Geometry types:
  Point      102654
  None       101647
Name: count, dtype: int64

```

```
CRS: EPSG:4326
```

```

# inspect missingness
missing_summary = pd.DataFrame({
    "nan_vals": data.isna().sum(),
    "Percentage": (data.isna().sum() / len(data)*100).round(2)
}).sort_values("Percentage", ascending=False)

print("\nMissingness by column:\n", missing_summary)

```

```
Missingness by column:
```

|                  | nan vals | Percentage |
|------------------|----------|------------|
| geom             | 204301   | 100.00     |
| businesssubtype  | 182748   | 89.45      |
| unitttype        | 155610   | 76.17      |
| unit             | 155345   | 76.04      |
| businessradename | 126897   | 62.11      |
| geo_point_2d     | 101647   | 49.75      |
| geometry         | 101647   | 49.75      |
| postalcode       | 95239    | 46.62      |
| house            | 94501    | 46.26      |
| street           | 94484    | 46.25      |
| feepaid          | 75420    | 36.92      |
| country          | 45005    | 22.03      |
| issueddate       | 28778    | 14.09      |
| expireddate      | 28755    | 14.07      |
| businessname     | 13483    | 6.60       |
| localarea        | 2714     | 1.33       |
| province         | 82       | 0.04       |

|                       |    |      |
|-----------------------|----|------|
| city                  | 58 | 0.03 |
| businessstype         | 0  | 0.00 |
| status                | 0  | 0.00 |
| licencerevisionnumber | 0  | 0.00 |
| licencenumber         | 0  | 0.00 |
| folderyear            | 0  | 0.00 |
| licencersn            | 0  | 0.00 |
| numberofemployees     | 0  | 0.00 |
| extractdate           | 0  | 0.00 |

## 1) A1 Analysis

The geom column is 100% missing so I'll drop this column

geometry/geo\_point\_2d is half missing so I'll drop those rows but figure out how many rows were lost

businessstype is 89.45% missing, so the industry\_label refinement will generally go back to business type

postalcode is 46.62% missing so it's not reliable as a backup for location indication

numberofemployees has 0% missing so it's all good.

feepaid is 36.92% missing so it's worth adding missing flag instead of assigning any blame to it

```
# dropping rows with completely empty geom column
data=data.drop(columns=["geom"])

# dealing with records with no geographic coords
n_before=len(data)
data=data[data.geometry.notna()].copy()
n_after=len(data)
print(f"dropped {n_before - n_after} rows without any geometry "
      f"({(n_before - n_after)/n_before*100:.2f}% of dataset)")

dropped 101647 rows without any geometry (49.75% of dataset)
```

## 2) A1 Analysis

The reason I drop these rows is because rows with no geometry can't be mapped or used in location clustering which will be needed for the next part (A2). I've also showed here exactly how many rows were dropped

```
# dealing with missing feepaid
data["feepaid_missing"]=data["feepaid"].isna()
```

### 3) A1 Analysis

Fee paid is a numeric field used for context and to filter. It is not used for the main geom clustering. Dropping rows here for missing data would also take away useful location data.

Instead I'll keep the rows and flag the missingness. I won't assign a fake number for the column since that could bias the end result.

```
# filter to sensible subset
print(data["status"].value_counts())
data=data[data["status"]=="Issued"].copy()
print(f"\nRows after filtering to issued:{len(data)}")
```

```
status
Issued          85950
Pending         5808
Gone Out of Business 4879
Inactive        4209
Cancelled       1808
Name: count, dtype: int64
```

Rows after filtering to issued:85950

### 4) A1 Analysis

I only want licences that are currently active. Pending, cancelled, or closed businesses aren't part of 'current' business activity. Mixing them in would make it harder to fairly compare which active businesses look unusual.

```
# consolidating businesstype
type_counts=data["businesstype"].value_counts()
type_pct =(type_counts/ type_counts.sum()*100)
cumulative=type_pct.cumsum()
print(type_counts.head(20))
print("\ncumulative % coverage by top n categories:")
print(cumulative.head(20))

# keeping however many categories cover 85% of records
# and bucketing the rest as "Other."
threshold = 85
top_categories =cumulative[cumulative<=threshold].index.tolist()
if len(top_categories)<len(cumulative):
    top_categories.append(cumulative.index[len(top_categories)])
print(f"\nkeeping {len(top_categories)}categories covering
~{threshold}% of data")

data["businesstype_grouped"]=data["businesstype"].where(data["business
type"].isin(top_categories), "Other")
print(data["businesstype_grouped"].value_counts())
```

|                                               |       |
|-----------------------------------------------|-------|
| businessstype                                 |       |
| Health Care Professionals and Services        | 11819 |
| Long-term Rental                              | 10265 |
| Retail Dealer                                 | 6142  |
| Legal Services                                | 5214  |
| Restaurant                                    | 4919  |
| Beauty Services                               | 3942  |
| Limited Service Food Establishment            | 3818  |
| Business Support Services                     | 2968  |
| Financial Services                            | 2402  |
| Consulting and Management Services            | 2354  |
| Real Estate Services                          | 2056  |
| Retail Dealer - Food                          | 1937  |
| General Contractor                            | 1790  |
| Association or Society                        | 1723  |
| Information Communication Technology          | 1707  |
| Wholesale Dealer - Non-Food                   | 1458  |
| Parking Area / Garage                         | 1194  |
| Health Care Facility                          | 1043  |
| Architectural and Engineering Services        | 983   |
| Non-Food Manufacturer Assembler and Processor | 973   |
| Name: count, dtype: int64                     |       |

cumulative % coverage by top n categories:

|                                               |           |
|-----------------------------------------------|-----------|
| businessstype                                 |           |
| Health Care Professionals and Services        | 13.751018 |
| Long-term Rental                              | 25.694008 |
| Retail Dealer                                 | 32.840023 |
| Legal Services                                | 38.906341 |
| Restaurant                                    | 44.629436 |
| Beauty Services                               | 49.215823 |
| Limited Service Food Establishment            | 53.657941 |
| Business Support Services                     | 57.111111 |
| Financial Services                            | 59.905759 |
| Consulting and Management Services            | 62.644561 |
| Real Estate Services                          | 65.036649 |
| Retail Dealer - Food                          | 67.290285 |
| General Contractor                            | 69.372891 |
| Association or Society                        | 71.377545 |
| Information Communication Technology          | 73.363583 |
| Wholesale Dealer - Non-Food                   | 75.059919 |
| Parking Area / Garage                         | 76.449098 |
| Health Care Facility                          | 77.662595 |
| Architectural and Engineering Services        | 78.806283 |
| Non-Food Manufacturer Assembler and Processor | 79.938336 |
| Name: count, dtype: float64                   |           |

keeping 27categories covering ~85% of data

|                       |       |
|-----------------------|-------|
| businessstype_grouped |       |
| Other                 | 12544 |

|                                               |       |
|-----------------------------------------------|-------|
| Health Care Professionals and Services        | 11819 |
| Long-term Rental                              | 10265 |
| Retail Dealer                                 | 6142  |
| Legal Services                                | 5214  |
| Restaurant                                    | 4919  |
| Beauty Services                               | 3942  |
| Limited Service Food Establishment            | 3818  |
| Business Support Services                     | 2968  |
| Financial Services                            | 2402  |
| Consulting and Management Services            | 2354  |
| Real Estate Services                          | 2056  |
| Retail Dealer - Food                          | 1937  |
| General Contractor                            | 1790  |
| Association or Society                        | 1723  |
| Information Communication Technology          | 1707  |
| Wholesale Dealer - Non-Food                   | 1458  |
| Parking Area / Garage                         | 1194  |
| Health Care Facility                          | 1043  |
| Architectural and Engineering Services        | 983   |
| Non-Food Manufacturer Assembler and Processor | 973   |
| Vehicle Repair Detailing and Washing Services | 894   |
| Food Manufacturer Assembler and Processor     | 751   |
| Photography Production and Rehearsal Studio   | 674   |
| Insurance Services                            | 617   |
| Mining Services                               | 607   |
| Fitness Centre                                | 587   |
| Liquor Establishment                          | 569   |
| Name: count, dtype: int64                     |       |

## A2 — Location-only clustering: K-means vs. DBSCAN

```
# extracting lat/lon from geometry
data["lon"]=data.geometry.x
data["lat"]=data.geometry.y
coords =data[["lat", "lon"]].dropna().copy()
print(f"rows used for clustering:{len(coords)}")

# standardizing coords to help K-means treat lat/lon on more
appropriate scales
scaler=StandardScaler()
coords_scaled=scaler.fit_transform(coords[["lat", "lon"]])

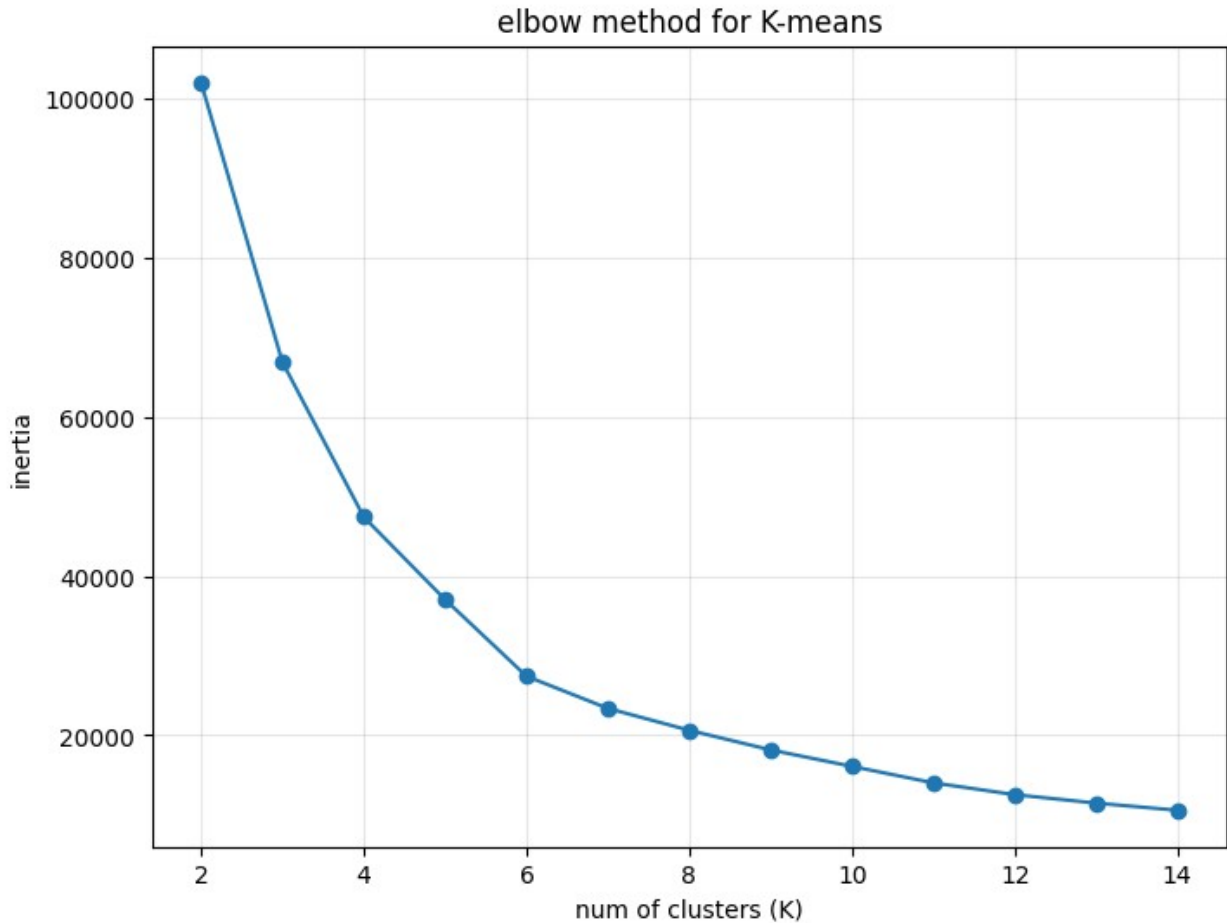
rows used for clustering:85950

# elbow method to choose K
inertias= []
Krange=range(2, 15)
for k in Krange:
    km=KMeans(n_clusters=k,random_state=42,n_init=10)
    km.fit(coords_scaled)
```

```

    inertias.append(km.inertia_)
plt.figure(figsize=(8,6))
plt.plot(list(Krange),inertias, marker="o")
plt.xlabel("num of clusters (K)")
plt.ylabel("inertia")
plt.title("elbow method for K-means")
plt.grid(True, alpha=0.3)
plt.show()

```



## 1) A2 Analysis

The elobow graph is showing 6 as the best K

```

# fitting with K
# graph shows visible bend at K=6
K = 6

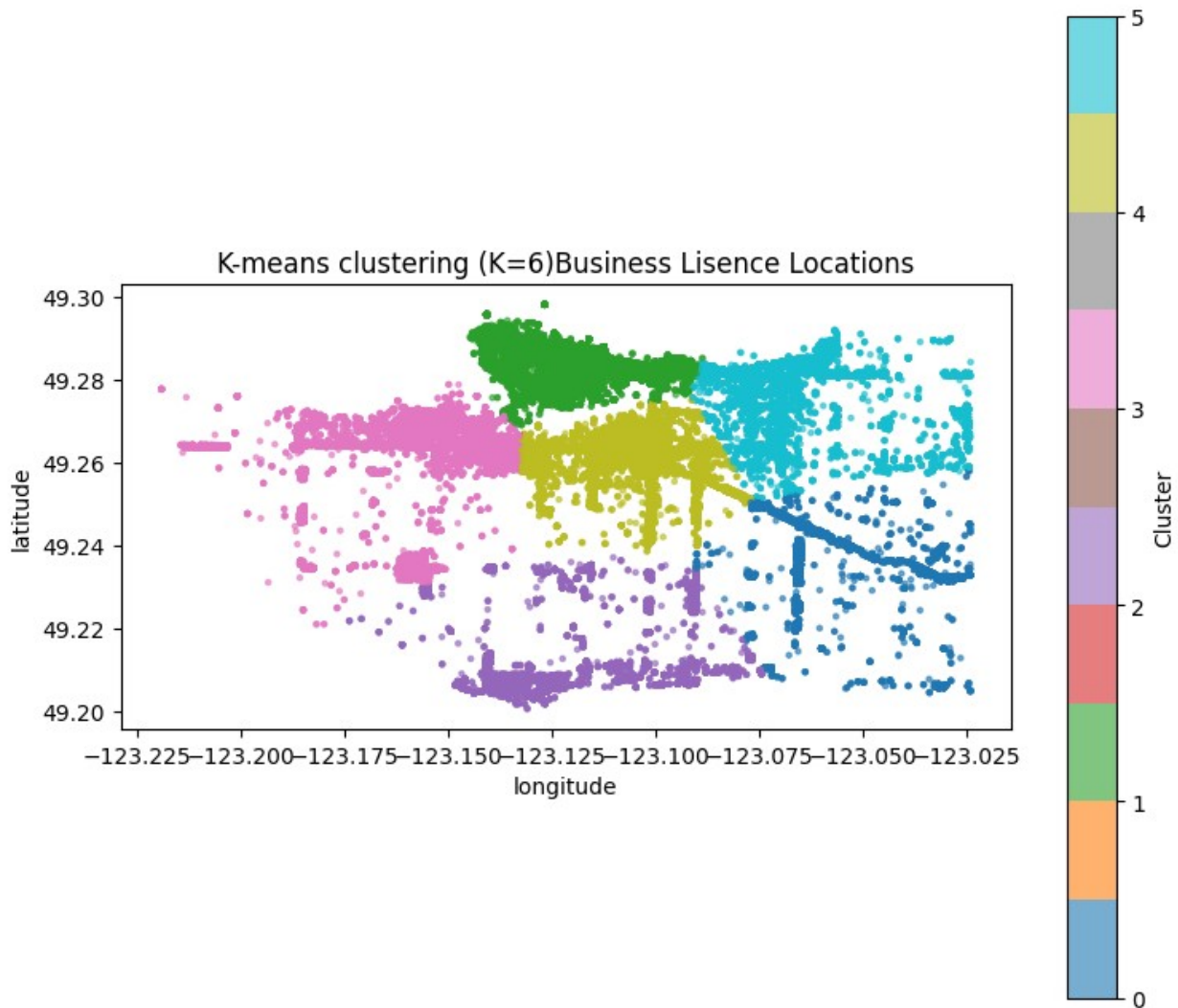
kmeans = KMeans(n_clusters=K, random_state=42, n_init=10)
coords["kmeans_cluster"] =kmeans.fit_predict(coords_scaled)
plt.figure(figsize=(9,8))

```

```

scatter=plt.scatter(
    coords["lon"], coords["lat"], c=coords["kmeans_cluster"], cmap=
    "tab10", s=5, alpha= 0.6)
plt.xlabel("longitude")
plt.ylabel("latitude")
plt.title(f"K-means clustering (K={K})Business Lisence Locations")
plt.colorbar(scatter, label= "Cluster")
plt.gca().set_aspect("equal")
plt.show()

```



```

# choosing eps/min_samples
dbscan = DBSCAN(eps=0.027,min_samples=20)
coords["dbscan_cluster"]=dbscan.fit_predict(coords_scaled)
n_clusters_dbscan=len(set(coords["dbscan_cluster"])) - (1 if -1 in
coords["dbscan_cluster"].values else 0)
n_noise =(coords["dbscan_cluster"]== -1).sum()

```

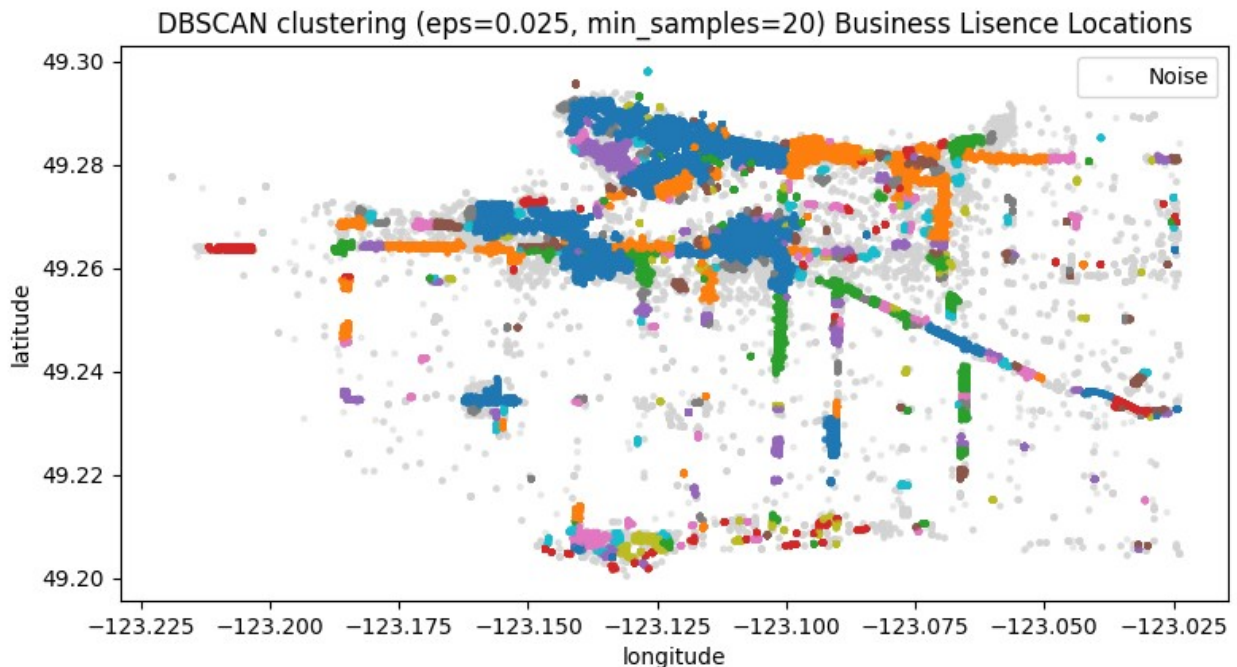
```

print(f"DBSCAN found {n_clusters_dbscan} clusters")
print(f"noise points: {n_noise} ({n_noise/len(coords)*100:.2f}%)")
plt.figure(figsize=(9, 8))

# plotting noise points initially so that actual clusters are clearly
# visible over top
noise_mask = coords["dbscan_cluster"]==-1
plt.scatter(coords.loc[noise_mask, "lon"], coords.loc[noise_mask,
"lat"],
            c="lightgray", s=5, alpha=0.4, label="Noise")
plt.scatter(coords.loc[~noise_mask, "lon"], coords.loc[~noise_mask,
"lat"],
            c=coords.loc[~noise_mask, "dbscan_cluster"], cmap="tab10", s=5,
alpha=0.6)
plt.xlabel("longitude")
plt.ylabel("latitude")
plt.title(f"DBSCAN clustering (eps=0.025, min_samples=20) Business
Licence Locations")
plt.gca().set_aspect("equal")
plt.legend()
plt.show()

```

DBSCAN found 429 clusters  
noise points: 7831(9.11%)



## 2) A2 Analysis

How do the K-means clusters map onto actual Vancouver geography — do they correspond to anything you'd recognize (downtown core, west side, etc.)?

The K-means map splits things into 6 chunks and they roughly match how someone would naturally divide the area. The clusters look like cleanly separated areas with easily discernable borders. This is because K-means doesn't know anything about real neighborhoods. It groups points that are physically close into equal-ish parts. It lines up with recognizable parts of the city but that's likely due to how the density of businesses is spread out in Vancouver (and not the algorithm understanding the geography)

## 3) A2 Analysis

How do the DBSCAN clusters compare?

DBSCAN gives a completely different outcome. It found 429 tinier clusters and about 9% that was noise and didn't fit into any of them.

The two methods are very different, K-means forces every single business into a fixed amount of groups, nothing is left out. DBSCAN only groups businesses if they are packed closely enough. If the area is not dense enough it categorizes it as noise instead of forcing it into a group (like K-means does).

Where do the two methods agree or disagree, and why — think about what each algorithm assumes about cluster shape and density, and what DBSCAN's "noise" points represent.

I can see that both methods are picking up on the same generally busy streets. There are matching areas of high business density in both of the plots.

Where they disagree is that K-means smooths everything into big simple chunks, whereas DBSCAN breaks them into much smaller, precise chunks. DBSCAN also shows businesses that don't fit into the clusters.

The noise points that aren't in the clusters are more isolated businesses. This could be helpful info for analysis since it tells us that the business is more alone and without many neighbors. It could mean there is something special or unusual about it. K-means cannot identify these types of points since it puts every single point into a cluster.

## A3 — Feature-based clustering: Size, Industry, Lifecycle

```
# engineering and scaling features
```

```
features = data.copy()
```

```
# size features
```

```
features["numberofemployees"] = features["numberofemployees"].astype(float)
```

```

at)
features["feepaid"]=features["feepaid"].astype(float)
features["feepaid"]=features["feepaid"].fillna(features["feepaid"].median())
features["feepaid_missing"]= data["feepaid"].isna().astype(int)

# lifecycle
features["issueddate"]=pd.to_datetime(features["issueddate"],
errors="coerce",utc=True).dt.tz_localize(None)
features["expireddate"]=pd.to_datetime(features["expireddate"],
errors="coerce",utc=True).dt.tz_localize(None)

# liscence duration in days
features["licence_duration_days"]=(features["expireddate"]-
features["issueddate"]).dt.days

# month issued, cyclically
features["issued_month"]=features["issueddate"].dt.month
features["issued_month_sin"]=np.sin(2*
np.pi*features["issued_month"]/12)
features["issued_month_cos"]=np.cos(2*np.pi*features["issued_month"]/
12)

# liscence revision num
features["licencerevisionnumber"]
=features["licencerevisionnumber"].astype(float)

# filling rows with missing duration or month with median instead of
dropping it
features["duration_missing"]=features["licence_duration_days"].isna().
astype(int)
features["licence_duration_days"]
=features["licence_duration_days"].fillna(features["licence_duration_d
ays"].median())
features["issued_month_sin"]=features["issued_month_sin"].fillna(0)
features["issued_month_cos"]=features["issued_month_cos"].fillna(0)

# industry
# swapping industry_label_grouped to try the finer version
industry_col="businesstype_grouped"
ohenc=OneHotEncoder(sparse_output=False, handle_unknown="ignore")
industry_encoded=ohenc.fit_transform(features[[industry_col]])
industry_cols=[f"industry_{c}" for c in ohenc.categories_[0] ]
industry_df=pd.DataFrame(industry_encoded, columns=industry_cols,
index=features.index)
print(f"industry one hot columns:  {len(industry_cols)}")

industry one hot columns:  28

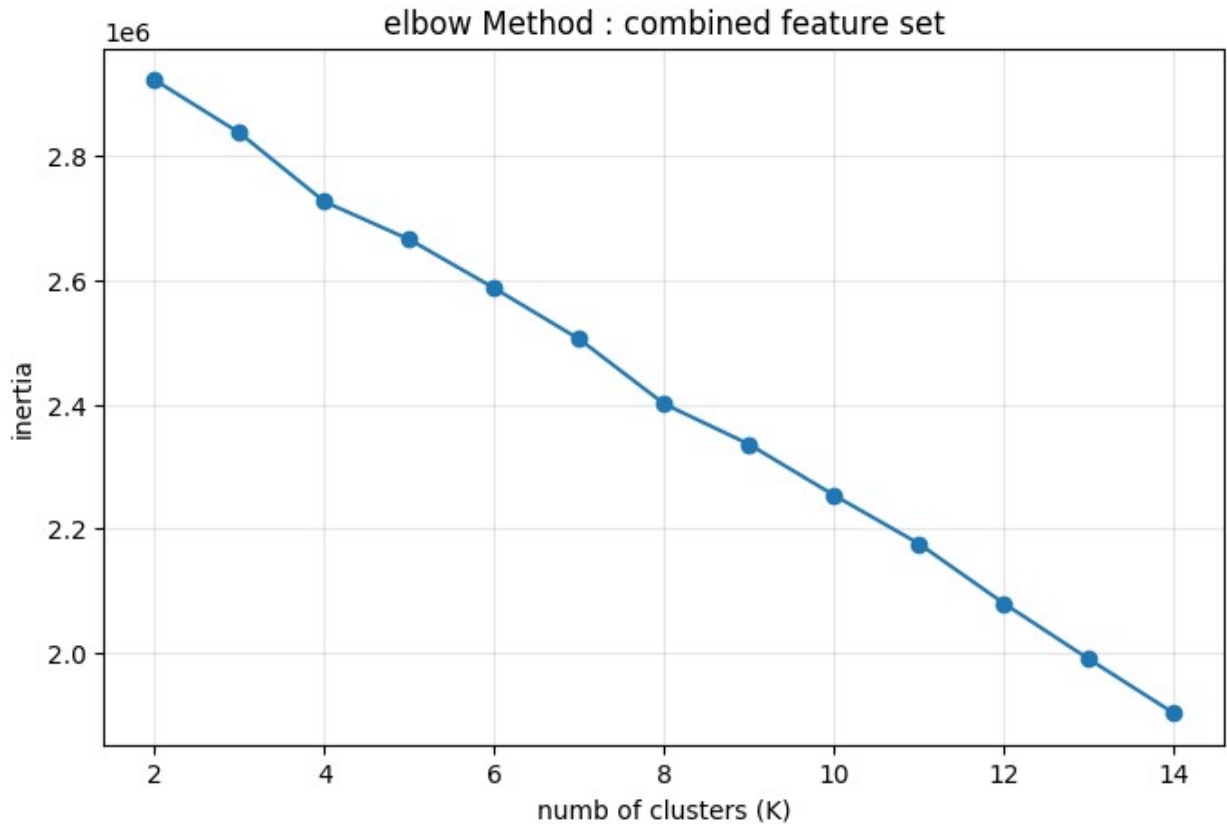
```

```

# combining and scaling features
numeric_features= ["numberofemployees", "feepaid", "feepaid_missing",
                  "licence_duration_days", "duration_missing",
                  "issued_month_sin","issued_month_cos",
                  "licencerevisionnumber",]
feature_df=pd.concat([features[numeric_features], industry_df], axis=
1)
scaler2=StandardScaler()
X_scaled=scaler2.fit_transform(feature_df)

# running simple Kmeans clustering and applying to PCA to project to 2D
and visualize clusters
# elbow method to choose K
inertias_features= []
K_range=range(2, 15)
for k in K_range:
    km=KMeans(n_clusters=k,random_state=42, n_init=10)
    km.fit(X_scaled)
    inertias_features.append(km.inertia_)
plt.figure(figsize=(8,5))
plt.plot(list(K_range), inertias_features, marker="o")
plt.xlabel("numb of clusters (K)")
plt.ylabel("inertia")
plt.title("elbow Method : combined feature set")
plt.grid(True, alpha=0.3)
plt.show()

```



```
# elbow bwgins to dip relatively more than other points at 7
K_FEATURES=7
kmeans_features=KMeans(n_clusters=K_FEATURES,random_state= 42,
n_init=10)
feature_df["cluster"]=kmeans_features.fit_predict(X_scaled)
print(feature_df["cluster"].value_counts())

cluster
0    36064
5    22833
3    17162
1     3818
2     2968
4     2354
6      751
Name: count, dtype: int64

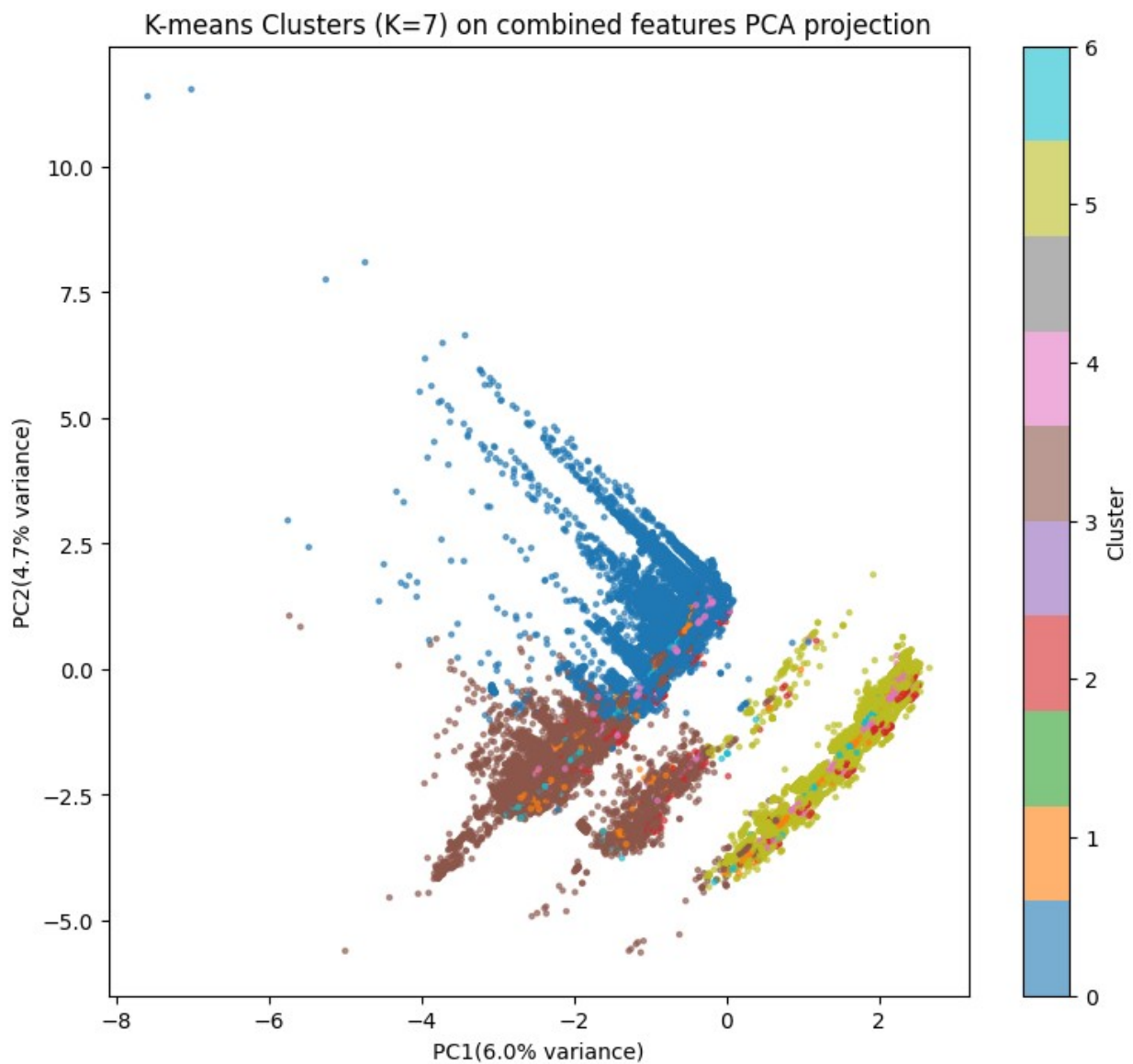
# PCA to 2D
pca=PCA(n_components=2, random_state=42)
X_pca=pca.fit_transform(X_scaled)
print(f"explained variance ratio:{pca.explained_variance_ratio_}")
print(f"total variance captured by 2 components:
{pca.explained_variance_ratio_.sum()*100:.1f}%")
plt.figure(figsize=(9, 8))
```

```

scatter = plt.scatter( X_pca[:, 0], X_pca[:, 1],
c=feature_df["cluster"], cmap="tab10", s=5, alpha=0.6)
plt.xlabel(f"PC1({pca.explained_variance_ratio_[0]*100:.1f}%
variance)")
plt.ylabel(f"PC2({pca.explained_variance_ratio_[1]*100:.1f}%
variance)")
plt.title(f"K-means Clusters (K={K_FEATURES}) on combined features PCA
projection")
plt.colorbar(scatter, label="Cluster")
plt.show()

```

explained variance ratio:[0.05969293 0.04718219]  
total variance captured by 2 components:10.7%



## 1) A3 Analysis

What do the resulting clusters seem to represent?

When I check the cluster sizes, they line almost exactly up almost exactly with the individual business types from earlier above. One cluster is all the Limited Service Food Establishments, another is all Business Support Services, and etc.

So instead of blending size, industry, and lifecycle like I was aiming for, Kmeans pretty much just re identified the industry categories I had.

This probably happened because industry was turned into 27 separate yes/no columns, whereas size and lifecycles only added a handful. With that amount of industry columns, they ended up dominating so KMeans grouped by industry mainly.

Does this differ from the purely geographic clustering in A2?

Yes, the A2 grouped business were put organized by their location, with no context of what kind of business they were. This one was meant to group them based on a mix of size/industry/and lifecycle (but mostly ended up showing industry as explained above)